

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: ITA 0614

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 20%

QUESTIONS: 2

Total Marks:20

Annexure A

Case Study

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Short Answer Test

```
1, # =====
# INTELLIGENT DISEASE DIAGNOSIS USING NAIVE BAYES
# Google Colab Program
# =====

# Step 1: Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)

# =====
# Step 2: Create Sample Patient Dataset
# =====

np.random.seed(42)

n = 300

data = {
    "Fever": np.random.randint(0, 2, n),
    "Cough": np.random.randint(0, 2, n),
    "Fatigue": np.random.randint(0, 2, n),
    "Headache": np.random.randint(0, 2, n),
    "Body_Pain": np.random.randint(0, 2, n),
    "Sore_Throat": np.random.randint(0, 2, n),
    "Breathing_Difficulty": np.random.randint(0, 2, n),
    "Age": np.random.randint(10, 80, n),
    "Previous_Illness": np.random.randint(0, 2, n)
}

df = pd.DataFrame(data)

# =====
```

Step 3: Generate Disease Labels

=====

```
def diagnose(row):
```

```
    score_flu = 0
```

```
    score_cold = 0
```

```
    score_pneumonia = 0
```

```
    # Flu-related symptoms
```

```
    if row["Fever"] == 1:
```

```
        score_flu += 2
```

```
    if row["Body_Pain"] == 1:
```

```
        score_flu += 2
```

```
    if row["Fatigue"] == 1:
```

```
        score_flu += 1
```

```
    if row["Headache"] == 1:
```

```
        score_flu += 1
```

```
    # Common cold
```

```
    if row["Cough"] == 1:
```

```
        score_cold += 2
```

```
    if row["Sore_Throat"] == 1:
```

```
        score_cold += 2
```

```
    if row["Headache"] == 1:
```

```
        score_cold += 1
```

```
    # Pneumonia
```

```
    if row["Breathing_Difficulty"] == 1:
```

```
        score_pneumonia += 3
```

```
    if row["Fever"] == 1:
```

```
        score_pneumonia += 2
```

```
    if row["Cough"] == 1:
```

```
        score_pneumonia += 2
```

```
    if row["Previous_Illness"] == 1:
```

```
        score_pneumonia += 1
```

```
    scores = {
```

```
        "Flu": score_flu,
```

```
        "Cold": score_cold,
```

```
        "Pneumonia": score_pneumonia
```

```
    }
```

```
return max(scores, key=scores.get)
```

```
df["Disease"] = df.apply(diagnose, axis=1)
```

```
# Display Dataset
print("PATIENT DATASET")
print("=" * 60)
print(df.head(10))
```

```
# =====
# Step 4: Display Disease Distribution
# =====
```

```
print("\nDISEASE DISTRIBUTION")
print("=" * 60)
print(df["Disease"].value_counts())
```

```
# =====
# Step 5: Select Features and Target
# =====
```

```
features = [
    "Fever",
    "Cough",
    "Fatigue",
    "Headache",
    "Body_Pain",
    "Sore_Throat",
    "Breathing_Difficulty",
    "Age",
    "Previous_Illness"
]
```

```
X = df[features]
y = df["Disease"]
```

```
# =====
# Step 6: Split Dataset
# =====
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
```

```

    random_state=42,
    stratify=y
)

# =====
# Step 7: Create Naive Bayes Model
# =====

model = GaussianNB()

model.fit(X_train, y_train)

# =====
# Step 8: Predict Test Data
# =====

y_pred = model.predict(X_test)

# =====
# Step 9: Calculate Accuracy
# =====

accuracy = accuracy_score(y_test, y_pred)

print("\nMODEL ACCURACY")
print("=" * 60)
print(f"Accuracy: {accuracy * 100:.2f}%")

# =====
# Step 10: Classification Report
# =====

print("\nCLASSIFICATION REPORT")
print("=" * 60)

print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)

# =====
# Step 11: Confusion Matrix

```

```

# =====

labels = ["Cold", "Flu", "Pneumonia"]

cm = confusion_matrix(
    y_test,
    y_pred,
    labels=labels
)

plt.figure(figsize=(7, 5))

plt.imshow(cm)

plt.title("Naive Bayes Disease Diagnosis")
plt.xlabel("Predicted Disease")
plt.ylabel("Actual Disease")

plt.xticks(
    range(len(labels)),
    labels
)

plt.yticks(
    range(len(labels)),
    labels
)

for i in range(len(labels)):
    for j in range(len(labels)):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.colorbar()
plt.show()

# =====
# Step 12: Probability Learning
# =====

print("\nLEARNED PRIOR PROBABILITIES")
print("=" * 60)

for disease, probability in zip(

```

```

        model.classes_,
        model.class_prior_
    ):
        print(
            f"P({disease}) = {probability:.4f}"
        )

# =====
# Step 13: Real-Time Patient Diagnosis
# =====

print("\nNEW PATIENT DETAILS")
print("=" * 60)

fever = int(input("Fever (1=Yes, 0=No): "))
cough = int(input("Cough (1=Yes, 0=No): "))
fatigue = int(input("Fatigue (1=Yes, 0=No): "))
headache = int(input("Headache (1=Yes, 0=No): "))
body_pain = int(input("Body Pain (1=Yes, 0=No): "))
sore_throat = int(input("Sore Throat (1=Yes, 0=No): "))
breathing = int(
    input("Breathing Difficulty (1=Yes, 0=No): ")
)
age = int(input("Age: "))
previous_illness = int(
    input("Previous Illness (1=Yes, 0=No): ")
)

# Create new patient record
new_patient = pd.DataFrame({
    "Fever": [fever],
    "Cough": [cough],
    "Fatigue": [fatigue],
    "Headache": [headache],
    "Body_Pain": [body_pain],
    "Sore_Throat": [sore_throat],
    "Breathing_Difficulty": [breathing],
    "Age": [age],
    "Previous_Illness": [previous_illness]
})

# =====
# Step 14: Predict Disease
# =====

prediction = model.predict(new_patient)[0]

```

```
probabilities = model.predict_proba(  
    new_patient  
)  
[0]
```

```
print("\nDIAGNOSIS RESULT")  
print("=" * 60)
```

```
print("Most Probable Disease:", prediction)
```

```
# =====  
# Step 15: Display Posterior Probabilities  
# =====
```

```
print("\nDISEASE PROBABILITIES")  
print("=" * 60)
```

```
for disease, probability in zip(  
    model.classes_,  
    probabilities  
):  
    print(  
        f"{disease}: {probability * 100:.2f}%"  
    )
```

```
# =====  
# Step 16: Probability Visualization  
# =====
```

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(  
    model.classes_,  
    probabilities * 100  
)
```

```
plt.xlabel("Disease")  
plt.ylabel("Probability (%)")  
plt.title("Bayesian Disease Prediction")
```

```
plt.ylim(0, 100)
```

```
plt.show()
```

```
# =====  
# Step 17: Show Conditional Feature Information  
# =====
```



```
print("\nFEATURE PROBABILITY INFORMATION")
print("=" * 60)
```

```
for i, feature in enumerate(features):
```

```
    print(f"\nFeature: {feature}")
```

```
    for j, disease in enumerate(model.classes_):
```

```
        mean = model.theta_[j][i]
        variance = model.var_[j][i]
```

```
        print(
            f"{disease}: "
            f"Mean={mean:.3f}, "
            f"Variance={variance:.3f}"
        )
```

PATIENT DATASET

```
=====
Fever Cough Fatigue Headache Body_Pain Sore_Throat \
0 0 0 0 0 0 0
1 1 0 1 1 1 1
2 0 0 1 1 1 1
3 0 1 1 0 1 0
4 0 1 0 0 0 0
5 1 0 0 1 1 0
6 0 1 0 1 0 1
7 0 1 1 0 1 1
8 0 1 1 0 0 1
9 1 1 1 1 0 1
```

```
Breathing_Difficulty Age Previous_Illness Disease
0 0 22 0 Flu
1 0 22 0 Flu
2 0 27 0 Flu
3 1 41 0 Pneumonia
4 0 48 1 Pneumonia
5 0 55 0 Flu
6 1 38 0 Cold
7 0 71 1 Cold
8 1 71 1 Pneumonia
9 1 66 0 Pneumonia
```

DISEASE DISTRIBUTION

```
=====
Disease
Pneumonia 145
Flu 109
Cold 46
Name: count, dtype: int64
```

MODEL ACCURACY

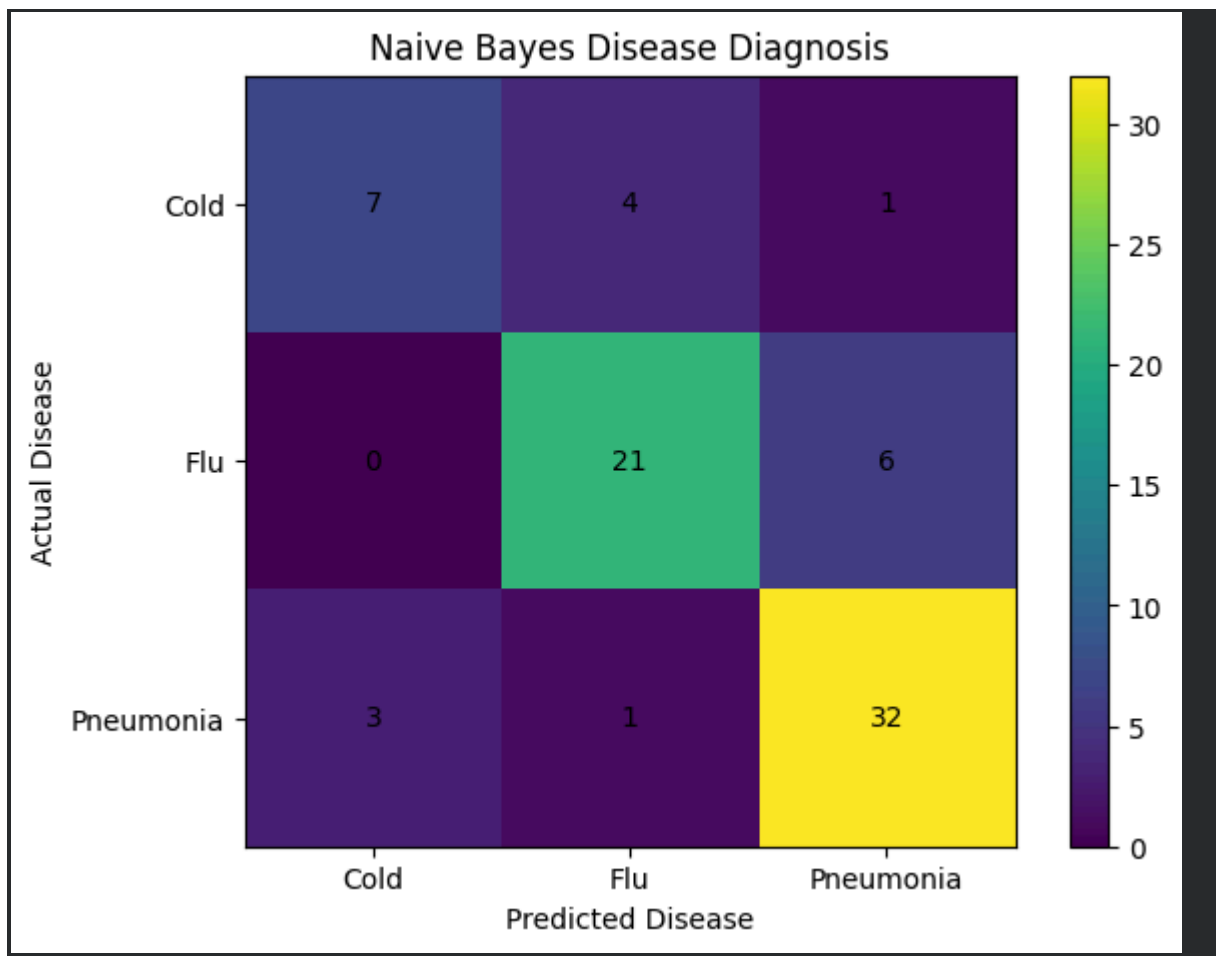
```
=====
Accuracy: 80.00%
```

CLASSIFICATION REPORT

```
=====
              precision  recall f1-score  support

Cold         0.70      0.58    0.64      12
Flu          0.81      0.78    0.79      27
Pneumonia    0.82      0.89    0.85      36

accuracy              0.80      75
macro avg          0.78    0.75    0.76      75
weighted avg       0.80    0.80    0.80      75
```



LEARNED PRIOR PROBABILITIES

```
=====
P(Cold) = 0.1511
P(Flu) = 0.3644
P(Pneumonia) = 0.4844
```

NEW PATIENT DETAILS

```
=====
2. # =====
```

```
# SPAM EMAIL DETECTION USING NAIVE BAYES
```

```
# MLE vs MDL FEATURE SELECTION
```

```

# Google Colab
# =====

# =====
# 1. IMPORT LIBRARIES
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    confusion_matrix
)

# =====
# 2. CREATE SAMPLE EMAIL DATASET
# =====

spam_messages = [
    "Congratulations you won a free lottery prize",
    "Win a free cash reward now",
    "Claim your free money today",
    "You have won a free gift voucher",
    "Limited time offer claim your prize",
    "Congratulations you are a lucky winner",
    "Get free money by clicking this link",
    "You won a cash prize click now",
    "Free vacation offer for selected winner",
    "Claim your reward before it expires",
    "Exclusive offer win cash today",
    "You have been selected for a free gift",
    "Click here to receive your prize",
    "Earn money quickly from home",
    "Get rich fast with this opportunity",

```

"Free coupon waiting for you",
"Congratulations claim your bonus",
"Urgent claim your free reward",
"Win amazing prizes today",
"Special offer free membership",
"You are our lucky winner",
"Free shopping voucher available",
"Claim cash bonus immediately",
"Win a brand new mobile phone",
"Limited offer get free rewards",
"You have won an exclusive prize",
"Free investment opportunity",
"Earn thousands from home",
"Click now for your free bonus",
"Congratulations free gift waiting"

]

ham_messages = [
"Please attend the meeting tomorrow",
"The project report is ready for review",
"Can you send me the assignment",
"Meeting is scheduled at ten AM",
"Please find the attached document",
"The team will discuss the project today",
"Your appointment is confirmed",
"Please submit the report by Friday",
"The presentation is ready",
"Can we discuss the project tomorrow",
"The class starts at nine AM",
"Please review the attached file",
"Your order has been shipped",
"The meeting room has been changed",
"Please call me when you are available",
"The project deadline is next week",
"Thank you for your email",
"I will send the documents today",
"Please confirm the meeting time",
"The report has been approved",
"We need to complete the project",
"Your appointment is tomorrow",
"Please check the attached report",
"The team meeting starts at two",
"Can you share the project details",
"The invoice has been received",

```

    "Please attend the training session",
    "The assignment deadline is Monday",
    "I have completed the requested work",
    "Let us discuss this in the meeting"
]

# Create DataFrame

spam_df = pd.DataFrame({
    "message": spam_messages,
    "label": 1
})

ham_df = pd.DataFrame({
    "message": ham_messages,
    "label": 0
})

df = pd.concat(
    [spam_df, ham_df],
    ignore_index=True
)

# Shuffle dataset

df = df.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

print("Dataset:")
print(df.head())

print("\nDataset Size:", len(df))

print("\nClass Distribution:")
print(df["label"].value_counts())

# =====
# 3. SPLIT DATASET
# =====

X_train_text, X_test_text, y_train, y_test = train_test_split(

```

```
    df["message"],
    df["label"],
    test_size=0.30,
    random_state=42,
    stratify=df["label"]
)
```

```
# =====
# 4. TEXT FEATURE EXTRACTION
# =====
```

```
vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english"
)
```

```
X_train = vectorizer.fit_transform(X_train_text)
```

```
X_test = vectorizer.transform(X_test_text)
```

```
print("\nNumber of Features:")
print(X_train.shape[1])
```

```
# =====
# 5. NAIVE BAYES CLASSIFIER
# =====
```

```
nb_model = MultinomialNB()
```

```
nb_model.fit(
    X_train,
    y_train
)
```

```
# =====
# 6. PREDICTION
# =====
```

```
y_pred = nb_model.predict(X_test)
```

```
y_probability = nb_model.predict_proba(X_test)
```

```
# =====  
# 7. PERFORMANCE EVALUATION  
# =====
```

```
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)
```

```
precision = precision_score(  
    y_test,  
    y_pred  
)
```

```
recall = recall_score(  
    y_test,  
    y_pred  
)
```

```
f1 = f1_score(  
    y_test,  
    y_pred  
)
```

```
print("\nMODEL PERFORMANCE")  
print("=" * 60)
```

```
print(f"Accuracy : {accuracy * 100:.2f}%")  
print(f"Precision : {precision * 100:.2f}%")  
print(f"Recall : {recall * 100:.2f}%")  
print(f"F1 Score : {f1 * 100:.2f}%")
```

```
# =====  
# 8. CLASSIFICATION REPORT  
# =====
```

```
print("\nCLASSIFICATION REPORT")  
print("=" * 60)
```

```
print(  
    classification_report(  

```

```

        y_test,
        y_pred,
        target_names=["Ham", "Spam"]
    )
)

# =====
# 9. CONFUSION MATRIX
# =====

cm = confusion_matrix(
    y_test,
    y_pred
)

plt.figure(figsize=(6, 5))

plt.imshow(cm)

plt.title("Spam Detection Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.xticks(
    [0, 1],
    ["Ham", "Spam"]
)

plt.yticks(
    [0, 1],
    ["Ham", "Spam"]
)

for i in range(2):
    for j in range(2):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

```



```
plt.colorbar()
```

```
plt.show()
```

```
# =====  
# 10. BAYES PROBABILITY FOR TEST EMAILS  
# =====
```

```
print("\nEXAMPLE PREDICTIONS")  
print("=" * 60)
```

```
for i in range(min(10, len(X_test_text))):
```

```
    message = X_test_text.iloc[i]
```

```
    prediction = y_pred[i]
```

```
    probabilities = y_probability[i]
```

```
    print("\nEmail:", message)
```

```
    if prediction == 1:  
        print("Prediction: SPAM")  
    else:  
        print("Prediction: HAM")
```

```
    print(  
        f"Ham Probability : {probabilities[0] * 100:.2f}%"  
    )
```

```
    print(  
        f"Spam Probability : {probabilities[1] * 100:.2f}%"  
    )
```

```
# =====  
# 11. MLE FEATURE SELECTION  
# =====  
#  
# MLE-style feature selection:  
# Select words according to their discriminative probability  
# in spam and ham messages.
```

```

#
# Here we calculate:
#
# P(word | spam)
# P(word | ham)
#
# and use the difference as a simple feature importance score.
# =====

feature_names = np.array(
    vectorizer.get_feature_names_out()
)

spam_count = np.asarray(
    X_train[y_train.values == 1].sum(axis=0)
).flatten()

ham_count = np.asarray(
    X_train[y_train.values == 0].sum(axis=0)
).flatten()

spam_total = spam_count.sum()
ham_total = ham_count.sum()

# Laplace smoothing

spam_probability = (
    spam_count + 1
) / (
    spam_total + len(feature_names)
)

ham_probability = (
    ham_count + 1
) / (
    ham_total + len(feature_names)
)

# MLE discriminative score

mle_score = np.abs(
    np.log(spam_probability / ham_probability)
)

```

```
mle_features = pd.DataFrame({
    "Word": feature_names,
    "MLE_Score": mle_score
})
```

```
mle_features = mle_features.sort_values(
    "MLE_Score",
    ascending=False
)
```

```
print("\nTOP MLE FEATURES")
print("=" * 60)
```

```
print(
    mle_features.head(20)
)
```

```
# =====
# 12. MDL-STYLE FEATURE SELECTION
# =====
#
# MDL = Minimum Description Length
#
# The basic idea is to select a compact set of features
# that provides good classification performance while
# reducing unnecessary model complexity.
#
# We test different vocabulary sizes and select the smallest
# vocabulary that gives strong validation performance.
# =====
```

```
vocabulary_sizes = [
    10,
    20,
    30,
    40,
    50,
    60,
    80,
    100
]
```

```
mdl_results = []
```

```
for vocab_size in vocabulary_sizes:
```

```
    vectorizer_mdl = CountVectorizer(  
        lowercase=True,  
        stop_words="english",  
        max_features=vocab_size  
    )
```

```
    X_train_mdl = vectorizer_mdl.fit_transform(  
        X_train_text  
    )
```

```
    X_test_mdl = vectorizer_mdl.transform(  
        X_test_text  
    )
```

```
    model_mdl = MultinomialNB()
```

```
    model_mdl.fit(  
        X_train_mdl,  
        y_train  
    )
```

```
    prediction_mdl = model_mdl.predict(  
        X_test_mdl  
    )
```

```
    accuracy_mdl = accuracy_score(  
        y_test,  
        prediction_mdl  
    )
```

```
# Simple MDL-style objective:  
# classification error + model complexity penalty
```

```
error = 1 - accuracy_mdl
```

```
complexity_penalty = (  
    vocab_size / 10000  
)
```

```
mdl_score = (  
    error + complexity_penalty
```

```

    )

    mdl_results.append([
        vocab_size,
        accuracy_mdl,
        mdl_score
    ])

mdl_df = pd.DataFrame(
    mdl_results,
    columns=[
        "Features",
        "Accuracy",
        "MDL_Score"
    ]
)

print("\nMDL FEATURE SELECTION RESULTS")
print("=" * 60)

print(mdl_df)

# =====
# 13. SELECT BEST MDL MODEL
# =====

best_mdl_row = mdl_df.loc[
    mdl_df["MDL_Score"].idxmin()
]

best_features = int(
    best_mdl_row["Features"]
)

print("\nBEST MDL FEATURE SIZE")
print("=" * 60)

print(
    "Number of selected features:",
    best_features
)

```

```
print(
    f"Accuracy: "
    f"{best_md1_row['Accuracy'] * 100:.2f}%"
)
```

```
print(
    f"MDL Score: "
    f"{best_md1_row['MDL_Score']:.4f}"
)
```

```
# =====
# 14. PLOT MLE FEATURE IMPORTANCE
# =====
```

```
top_mle = mle_features.head(15)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.barh(
    top_mle["Word"][::-1],
    top_mle["MLE_Score"][::-1]
)
```

```
plt.xlabel("MLE Discriminative Score")
```

```
plt.ylabel("Word")
```

```
plt.title(
    "Top Features Selected Using MLE"
)
```

```
plt.show()
```

```
# =====
# 15. PLOT MDL RESULTS
# =====
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(
    mdl_df["Features"],
    mdl_df["Accuracy"],

```

```

        marker="o"
    )

plt.xlabel("Number of Features")

plt.ylabel("Accuracy")

plt.title(
    "Feature Count vs Classification Accuracy"
)

plt.grid(True)

plt.show()

```

```

# =====
# 16. SAMPLE SIZE ANALYSIS
# =====

```

```

sample_sizes = [
    20,
    40,
    60,
    80,
    100,
    120,
    140
]

```

```

sample_accuracy = []

```

```

for size in sample_sizes:

```

```

    if size >= len(df):
        continue

```

```

    sample_df = df.sample(
        n=size,
        random_state=42
    )

```

```

    X_sample_train, X_sample_test, y_sample_train, y_sample_test = train_test_split(
        sample_df["message"],

```

```

    sample_df["label"],
    test_size=0.30,
    random_state=42,
    stratify=sample_df["label"]
)

sample_vectorizer = CountVectorizer(
    lowercase=True,
    stop_words="english"
)

X_sample_train_vec = sample_vectorizer.fit_transform(
    X_sample_train
)

X_sample_test_vec = sample_vectorizer.transform(
    X_sample_test
)

sample_model = MultinomialNB()

sample_model.fit(
    X_sample_train_vec,
    y_sample_train
)

sample_prediction = sample_model.predict(
    X_sample_test_vec
)

sample_acc = accuracy_score(
    y_sample_test,
    sample_prediction
)

sample_accuracy.append(
    sample_acc
)

# Plot sample size effect

plt.figure(figsize=(8, 5))

```



```
plt.plot(
    sample_sizes[:len(sample_accuracy)],
    np.array(sample_accuracy) * 100,
    marker="o"
)

plt.xlabel("Training Dataset Size")

plt.ylabel("Accuracy (%)")

plt.title(
    "Effect of Sample Size on Spam Classification"
)

plt.grid(True)

plt.show()
```

```
# =====
# 17. HYPOTHESIS SPACE / MODEL COMPLEXITY ANALYSIS
# =====
#
# We vary the number of features.
# More features = larger hypothesis space.
# Fewer features = simpler hypothesis space.
# =====
```

```
feature_counts = [
    10,
    20,
    30,
    40,
    50,
    60,
    80
]
```

```
complexity_accuracy = []
```

```
for feature_count in feature_counts:
```

```
    complexity_vectorizer = CountVectorizer(
        lowercase=True,
```

```

        stop_words="english",
        max_features=feature_count
    )

X_train_complexity = (
    complexity_vectorizer.fit_transform(
        X_train_text
    )
)

X_test_complexity = (
    complexity_vectorizer.transform(
        X_test_text
    )
)

complexity_model = MultinomialNB()

complexity_model.fit(
    X_train_complexity,
    y_train
)

complexity_prediction = (
    complexity_model.predict(
        X_test_complexity
    )
)

complexity_acc = accuracy_score(
    y_test,
    complexity_prediction
)

complexity_accuracy.append(
    complexity_acc
)

# Plot hypothesis space effect

plt.figure(figsize=(8, 5))

plt.plot(

```

```

    feature_counts,
    np.array(complexity_accuracy) * 100,
    marker="o"
)

plt.xlabel(
    "Number of Features / Hypothesis Space Size"
)

plt.ylabel("Accuracy (%)")

plt.title(
    "Hypothesis Space vs Classification Accuracy"
)

plt.grid(True)

plt.show()

```

```

# =====
# 18. BAYES OPTIMAL CLASSIFIER CONCEPT
# =====

```

```

print("\nBAYES OPTIMAL CLASSIFIER")
print("=" * 60)

```

```

print("""
The Bayes Optimal Classifier chooses the class with
the highest posterior probability:

```

$P(\text{Class} \mid \text{Email})$

For each email:

1. Calculate probability of HAM.
2. Calculate probability of SPAM.
3. Compare both probabilities.
4. Select the class with the highest posterior probability.

In this implementation, Naive Bayes approximates the Bayes Optimal decision by assuming conditional independence between words.

```

""")

```

```
# =====  
# 19. FINAL COMPARISON  
# =====
```

```
print("\nFINAL ANALYSIS")  
print("=" * 60)
```

```
print("""  
MLE:
```

- Estimates probabilities directly from observed data.
- Simple and computationally efficient.
- Can select highly discriminative words.
- May select many features.

```
MDL:
```

- Balances model accuracy and model complexity.
- Prefers a compact feature representation.
- Can reduce unnecessary features.
- May slightly reduce accuracy while improving simplicity.

```
Sample Size:
```

- Small datasets may produce unreliable probability estimates.
- Larger datasets generally provide better probability estimates.
- More training data usually improves generalization.

```
Hypothesis Space:
```

- A very small hypothesis space may underfit.
- A very large hypothesis space can increase complexity.
- An appropriate feature size provides a good balance.

```
Naive Bayes:
```

- Fast and suitable for large numbers of emails.
- Works well with text classification.
- Requires relatively little training data.
- Assumes conditional independence between features.

```
Bayes Optimal Classifier:
```

- Theoretically chooses the class with the highest posterior probability.
- Provides the ideal decision rule when the true probability distribution is known.
- In practice, the true distribution is unknown, so Naive Bayes

is commonly used as an efficient approximation.

""")

Dataset:

```

      message label
0  Congratulations you won a free lottery prize    1
1  Congratulations you are a lucky winner         1
2  Your appointment is confirmed                 0
3  The project deadline is next week             0
4  Earn money quickly from home                  1
```

Dataset Size: 60

Class Distribution:

label

1 30

0 30

Name: count, dtype: int64

Number of Features:

74

MODEL PERFORMANCE

=====

Accuracy : 94.44%

Precision : 100.00%

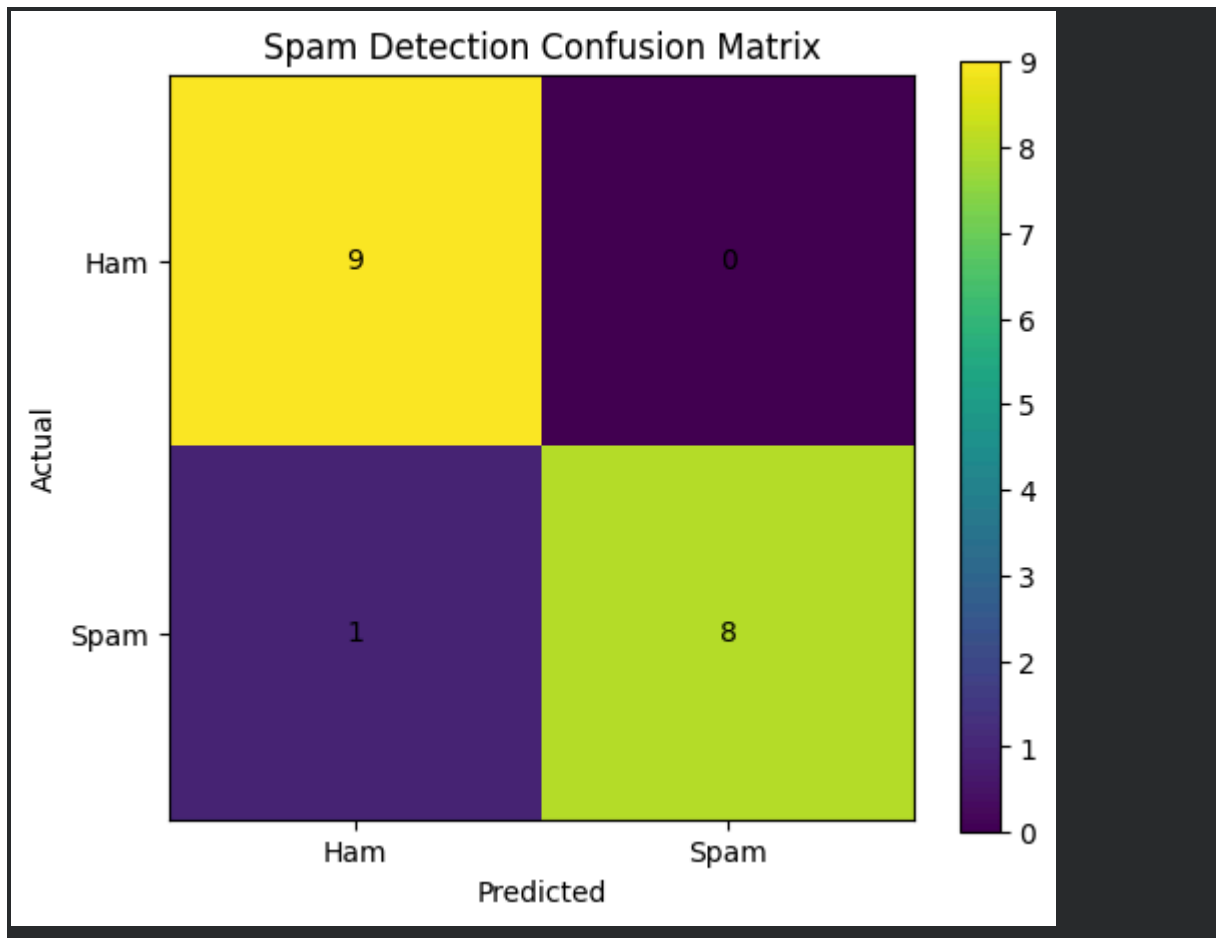
Recall : 88.89%

F1 Score : 94.12%

CLASSIFICATION REPORT

=====

	precision	recall	f1-score	support
Ham	0.90	1.00	0.95	9
Spam	1.00	0.89	0.94	9
accuracy			0.94	18
macro avg	0.95	0.94	0.94	18
weighted avg	0.95	0.94	0.94	18



EXAMPLE PREDICTIONS

=====

Email: Congratulations claim your bonus

Prediction: SPAM

Ham Probability : 3.05%

Spam Probability : 96.95%

Email: Congratulations free gift waiting

Prediction: SPAM

Ham Probability : 1.69%

Spam Probability : 98.31%

Email: The class starts at nine AM

Prediction: HAM

Ham Probability : 50.00%

Spam Probability : 50.00%

Email: The team meeting starts at two

Prediction: HAM

Ham Probability : 93.44%

Spam Probability : 6.56%

Email: Special offer free membership

Prediction: SPAM

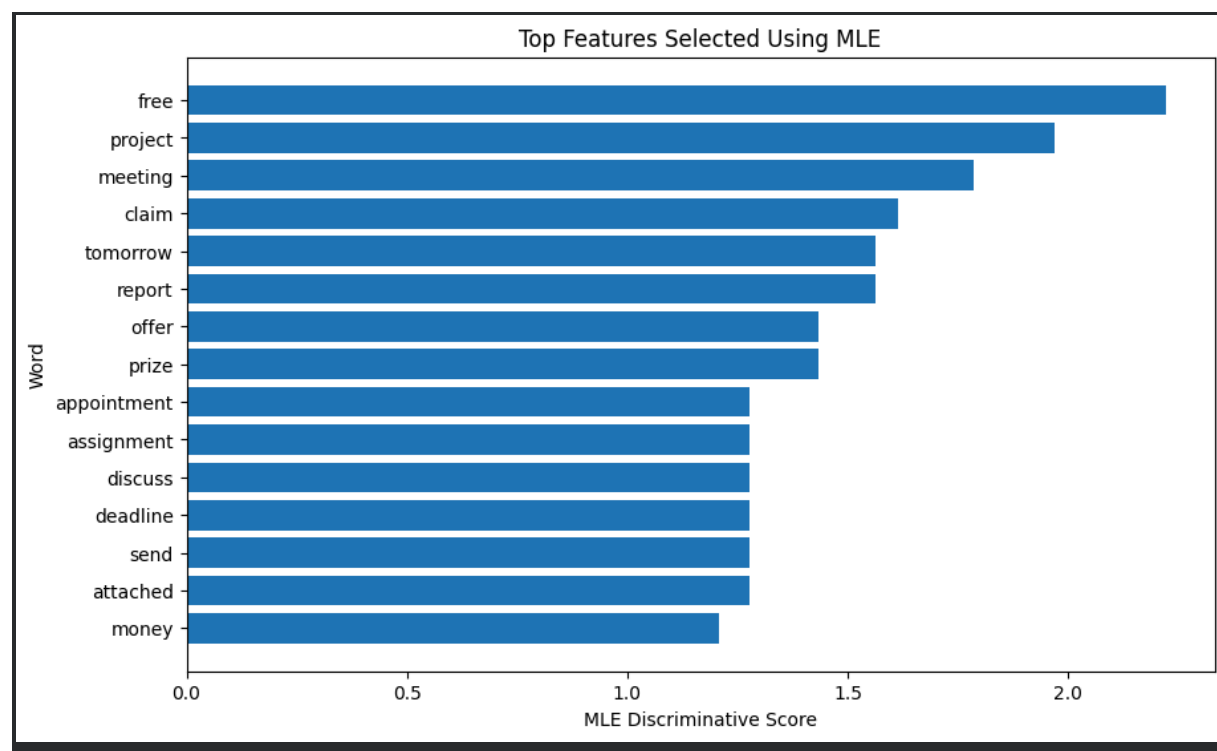
MDL FEATURE SELECTION RESULTS

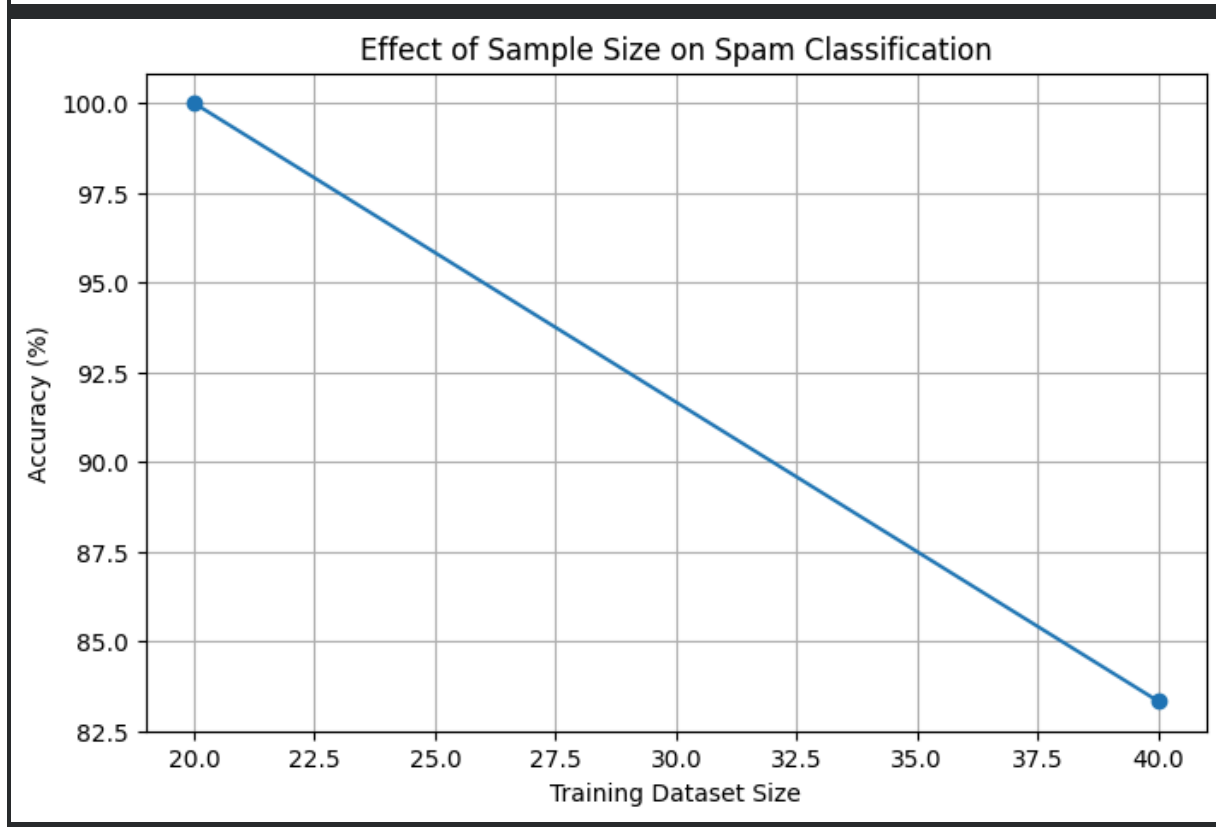
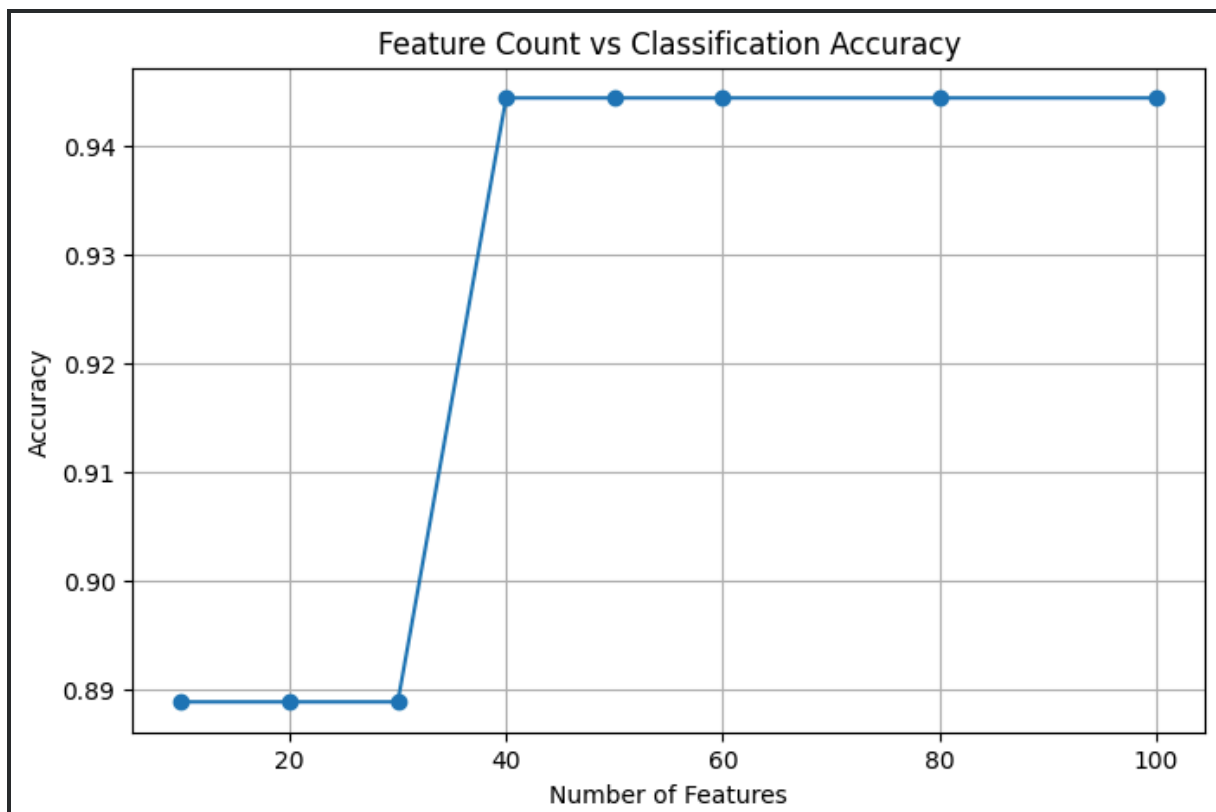
=====			
Features	Accuracy	MDL_Score	
0	10	0.888889	0.112111
1	20	0.888889	0.113111
2	30	0.888889	0.114111
3	40	0.944444	0.059556
4	50	0.944444	0.060556
5	60	0.944444	0.061556
6	80	0.944444	0.063556
7	100	0.944444	0.065556

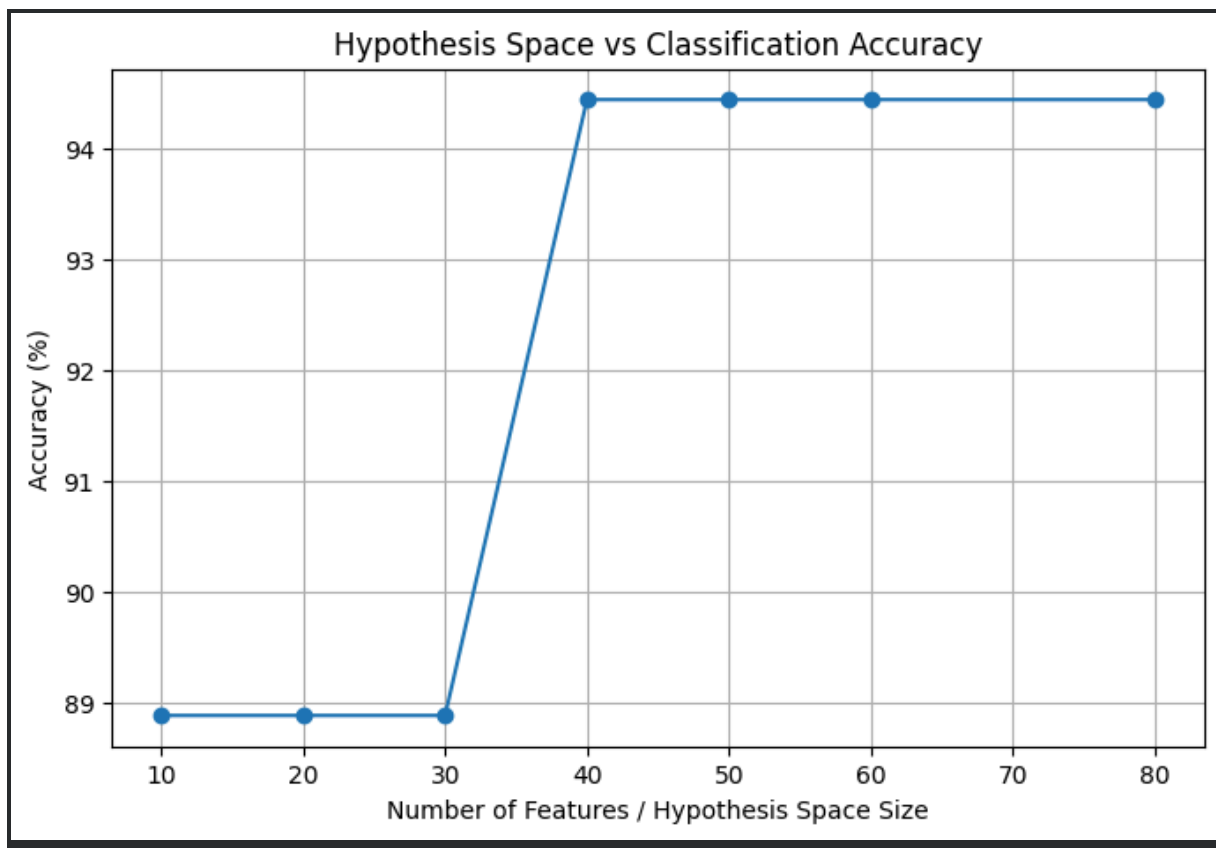
BEST MDL FEATURE SIZE

=====

Number of selected features: 40
Accuracy: 94.44%
MDL Score: 0.0596







BAYES OPTIMAL CLASSIFIER

=====

The Bayes Optimal Classifier chooses the class with the highest posterior probability:

$P(\text{Class} \mid \text{Email})$

For each email:

1. Calculate probability of HAM.
2. Calculate probability of SPAM.
3. Compare both probabilities.
4. Select the class with the highest posterior probability.

In this implementation, Naive Bayes approximates the Bayes Optimal decision by assuming conditional independence between words.

FINAL ANALYSIS

=====

MLE:

- Estimates probabilities directly from observed data.
- Simple and computationally efficient.
- Can select highly discriminative words.

- May select many features.

MDL:

- Balances model accuracy and model complexity.
- Prefers a compact feature representation.
- Can reduce unnecessary features.
- May slightly reduce accuracy while improving simplicity.

Sample Size:

- Small datasets may produce unreliable probability estimates.
- Larger datasets generally provide better probability estimates.
- More training data usually improves generalization.

Hypothesis Space:

- A very small hypothesis space may underfit.
- A very large hypothesis space can increase complexity.
- An appropriate feature size provides a good balance.

Naive Bayes:

- Fast and suitable for large numbers of emails.
- Works well with text classification.
- Requires relatively little training data.
- Assumes conditional independence between features.

Bayes Optimal Classifier:

- Theoretically chooses the class with the highest posterior probability.
- Provides the ideal decision rule when the true probability distribution is known.
- In practice, the true distribution is unknown, so Naive Bayes is commonly used as an efficient approximation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand